

FinPay

Distributed Wallet & Payment Processing Platform

Engineering Architecture & Design Document

Version 1.0 | Classification: Internal | Status: Pre-Implementation Review

1. Executive Summary

FinPay is a distributed wallet and payment processing platform built as a learning-focused engineering project to demonstrate how modern fintech systems reliably move money at scale. It is not a payment gateway, not a banking application, and not a commercial product. It is a structured, production-style backend system designed to explore the engineering challenges that underpin real-world money transfer systems.

What FinPay Is

FinPay gives users the ability to create accounts, create wallets, view balances, transfer money to other users, view transaction history, receive email notifications, and explore spending analytics. The platform is entirely backend-focused. There is no frontend. Every interaction happens via REST APIs.

Why It Exists

Most tutorials and learning projects treat payment systems as CRUD applications. A user sends money, the database is updated, done. Real payment infrastructure is fundamentally different. It must handle concurrent requests without double-spending, duplicate submissions without double-processing, network failures without data loss, partial failures without corrupt state, and eventual consistency without violating financial invariants.

FinPay exists to teach these principles through direct implementation using a production-grade technology stack.

What Engineering Problems It Solves

- Concurrent wallet debits that could cause negative balances
- Duplicate transfer requests submitted by retry-happy clients
- Partial failures mid-transfer that leave sender debited but receiver never credited
- Queue failures that drop payment jobs silently
- Notification failures that must not block payment completion
- Analytics that must not slow down the critical payment path
- Authentication vulnerabilities including token theft and session replay

Who Would Use Such a System

Any engineer preparing to work on fintech infrastructure, payment platforms, wallets, or distributed systems at companies like Razorpay, Paytm, Stripe, Adyen, PayPal, or any bank's digital arm would benefit from studying and building FinPay. The architecture directly mirrors production patterns used in real financial systems.

2. Product Vision

2.1 Business Perspective

From a business standpoint, FinPay solves the problem of reliable value transfer between users. The core proposition is simple: if User A has money in their wallet, they should be able to send it to User B, and the system must guarantee that the money leaves A's account exactly once and arrives in B's account exactly once — even in the presence of failures.

This sounds trivial but is the hardest problem in payments. Every major payment company has invested enormous engineering effort into solving exactly this problem.

2.2 Engineering Perspective

From an engineering standpoint, FinPay is a vehicle for understanding:

- How to design microservices that own clearly bounded data domains
- How to use queues to decouple critical paths from non-critical paths
- How distributed locking prevents race conditions on shared financial state
- How idempotency prevents duplicate operations from causing incorrect outcomes
- How the Saga pattern coordinates multi-step transactions with compensation logic
- How event-driven architecture enables real-time analytics without blocking payment flows

2.3 System Goals

Goal	Description
Reliability	Every transfer either completes fully or rolls back fully. No partial states persist.
Idempotency	Submitting the same transfer request multiple times produces the same outcome as submitting it once.
Concurrency Safety	Multiple simultaneous transfers involving the same wallet never cause incorrect balances.
Decoupled Processing	Notifications and analytics never block or slow the payment critical path.
Observability	Every request is traceable. Every failure is logged. Every queue is monitored.
Scalability	Workers, services, and queues can scale independently to handle load spikes.

2.4 Non-Goals

- FinPay does not integrate with any external bank, payment gateway, or card network

- FinPay does not implement regulatory compliance (KYC, AML, PCI-DSS)
- FinPay does not include a frontend UI
- FinPay does not handle multi-currency conversion
- FinPay does not target production deployment with real money
- FinPay does not implement fraud detection

3. Domain Modeling

The domain model defines the core entities in FinPay, their attributes, and their relationships. These entities map directly to MongoDB collections.

3.1 User

A User represents a registered person in the system. The User entity owns identity and authentication state. It does not own money — money lives in a Wallet.

Field	Description
_id	MongoDB ObjectId, primary key
name	Full name of the user
email	Unique email, used for login and notifications
passwordHash	Bcrypt hash of the user's password
status	active suspended deleted
createdAt	Account creation timestamp
updatedAt	Last modification timestamp

3.2 Wallet

A Wallet represents a named store of value owned by a User. A User can have exactly one wallet in the current scope. The Wallet tracks the current balance and enforces that balance never goes negative.

Field	Description
_id	MongoDB ObjectId, primary key
userId	Reference to User — the owner
balance	Current balance in paisa (integer, never float)
currency	ISO 4217 code, e.g. INR
status	active frozen closed
version	Optimistic lock version counter
createdAt / updatedAt	Timestamps

Balance is stored as an integer in the smallest currency unit (paisa for INR) to avoid floating-point errors.

3.3 Ledger Entry

A Ledger Entry is an immutable record of every credit or debit against a Wallet. Every balance change creates a ledger entry. Ledger entries are the source of truth for financial history. The current balance on a Wallet is a cached sum of all ledger entries.

Field	Description
_id	ObjectId
walletId	Which wallet was affected
transactionId	The transaction that caused this entry
type	credit debit
amount	Amount in paisa
balanceBefore	Balance snapshot before this operation
balanceAfter	Balance snapshot after this operation
description	Human-readable note
createdAt	Immutable creation timestamp

3.4 Transaction

A Transaction represents a transfer of value from one Wallet to another. It is the orchestration record. Its status tracks the progression through the payment lifecycle.

Field	Description
_id	ObjectId
idempotencyKey	Client-supplied unique key, ensures deduplication
senderId / senderWalletId	Who is sending
receiverId / receiverWalletId	Who is receiving
amount	Transfer amount in paisa
currency	Currency code
status	PENDING PROCESSING COMPLETED FAILED ROLLED_BACK
jobId	BullMQ job ID for the queued payment job
failureReason	Error message if status is FAILED
createdAt / updatedAt / completedAt	Timestamps

3.5 Notification

Field	Description
_id	ObjectId

Field	Description
userId	Recipient
type	payment_sent payment_received login_alert
channel	email
payload	Embedded object with email subject, body, metadata
status	PENDING SENT FAILED
attempts	Number of delivery attempts
createdAt / sentAt	Timestamps

3.6 Analytics Event

Field	Description
_id	ObjectId
eventType	transaction.completed transaction.failed
userId	Associated user
walletId	Associated wallet
amount	Transaction amount
period	YYYY-MM string for monthly aggregation
metadata	Free-form additional context
processedAt	When the analytics service consumed this event

3.7 Entity Relationships

```

User ————— has one ————— Wallet
Wallet ————— has many ————— LedgerEntry
Transaction — belongs to ————— Wallet (sender)
Transaction — belongs to ————— Wallet (receiver)
Transaction — produces ————— LedgerEntry (debit)
Transaction — produces ————— LedgerEntry (credit)
Transaction — triggers ————— Notification (*2, sender + receiver)
Transaction — triggers ————— AnalyticsEvent
User ————— has many ————— Notification
User ————— has many ————— RefreshToken

```

4. Functional Requirements

4.1 Authentication

- Users can register with name, email, and password
- Users can log in and receive a short-lived JWT access token and a long-lived refresh token
- Users can refresh their access token using a valid refresh token
- Refresh tokens rotate on every use — old token is invalidated
- Users can log out, which invalidates the current refresh token
- Access tokens are stateless and validated via JWT signature on every request
- Passwords are hashed with bcrypt before storage

4.2 Wallet Management

- An authenticated user can create a wallet
- A user can view their own wallet balance
- A user cannot view another user's wallet balance directly
- Wallet balance is always returned as the current confirmed balance

4.3 Money Transfers

- An authenticated user can initiate a transfer to another user by email or userId
- The transfer amount must be a positive integer (in paisa)
- The sender must have sufficient balance
- Transfers are idempotent — submitting the same idempotencyKey twice returns the original result
- Transfer processing is asynchronous — the API returns immediately with PENDING status
- Users can query the status of a transfer by transaction ID

4.4 Transaction History

- Users can view their full transaction history
- History is paginated
- Each record shows: direction (sent/received), amount, counterparty, status, timestamp

4.5 Notifications

- Sender receives an email notification when a transfer is successfully processed
- Receiver receives an email notification when money is credited to their wallet
- Failed transfers trigger a failure notification to the sender
- Notification history is stored and queryable

4.6 Analytics

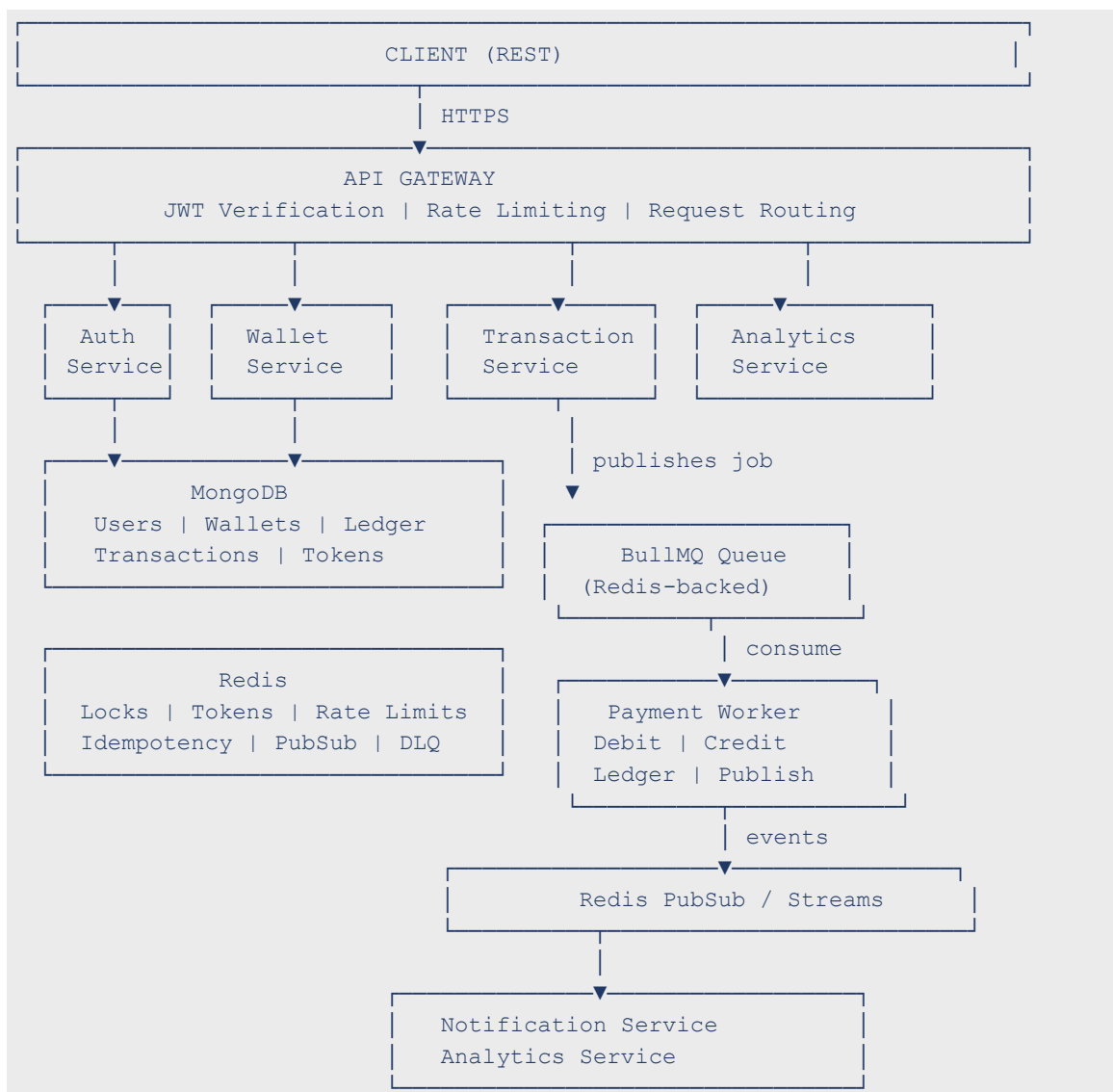
- Users can query their total money sent and received over a time period
- Aggregated spending summaries are available by month
- Analytics are eventually consistent — they may lag behind real-time by seconds to minutes

5. Non-Functional Requirements

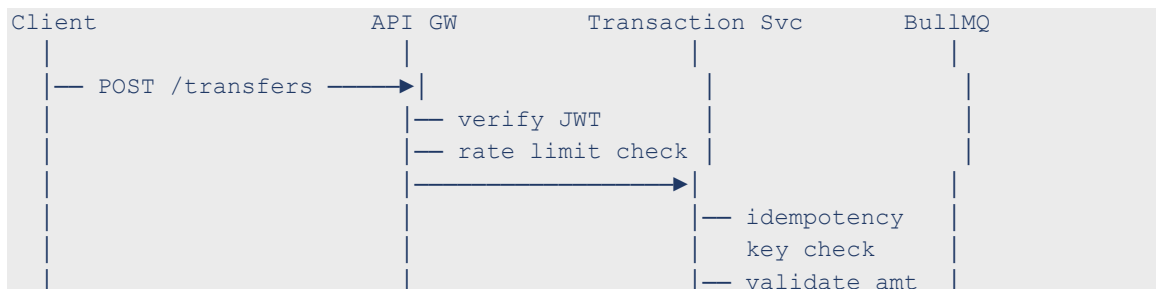
Requirement	Target / Approach
Scalability	Each service scales horizontally. Payment workers scale independently by adding more consumers. Redis handles distributed state.
Availability	Payment-critical path is highly available. Notification and analytics failures are isolated and do not affect payment availability.
Reliability	At-least-once delivery guaranteed via BullMQ. Idempotency prevents duplicate effects. Saga pattern handles rollbacks.
Consistency	Wallet balances are strongly consistent within a request (distributed lock). Analytics are eventually consistent.
Fault Tolerance	Workers retry failed jobs with exponential backoff. Dead Letter Queues capture permanently failed jobs. Services fail independently.
Performance	Transfer initiation p99 < 200ms. Queue processing p99 < 2s. Read APIs p99 < 100ms.
Security	All endpoints require JWT. Refresh tokens are rotated and stored securely. Passwords bcrypt-hashed. Rate limiting on auth endpoints.
Maintainability	Monorepo with clear service boundaries. Pino structured logging with correlation IDs. Each service independently deployable via Docker.
Observability	Every request has a traceId. Queue metrics visible via Bull Board. Logs are machine-parseable JSON.

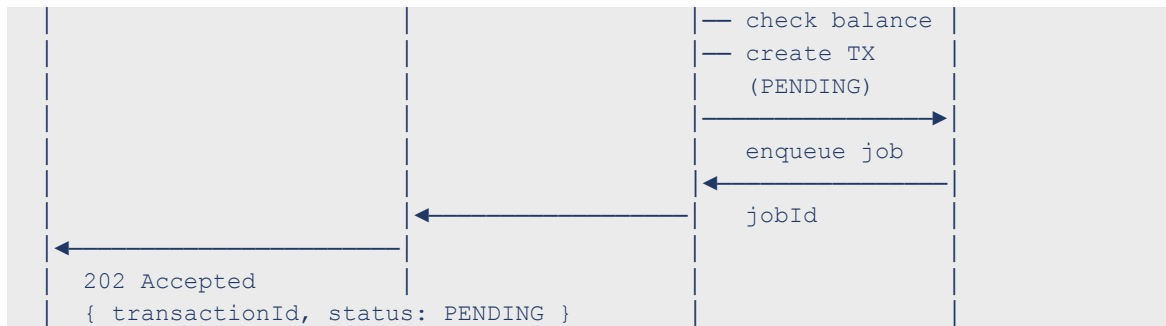
6. Complete System Architecture

6.1 High-Level Architecture Diagram

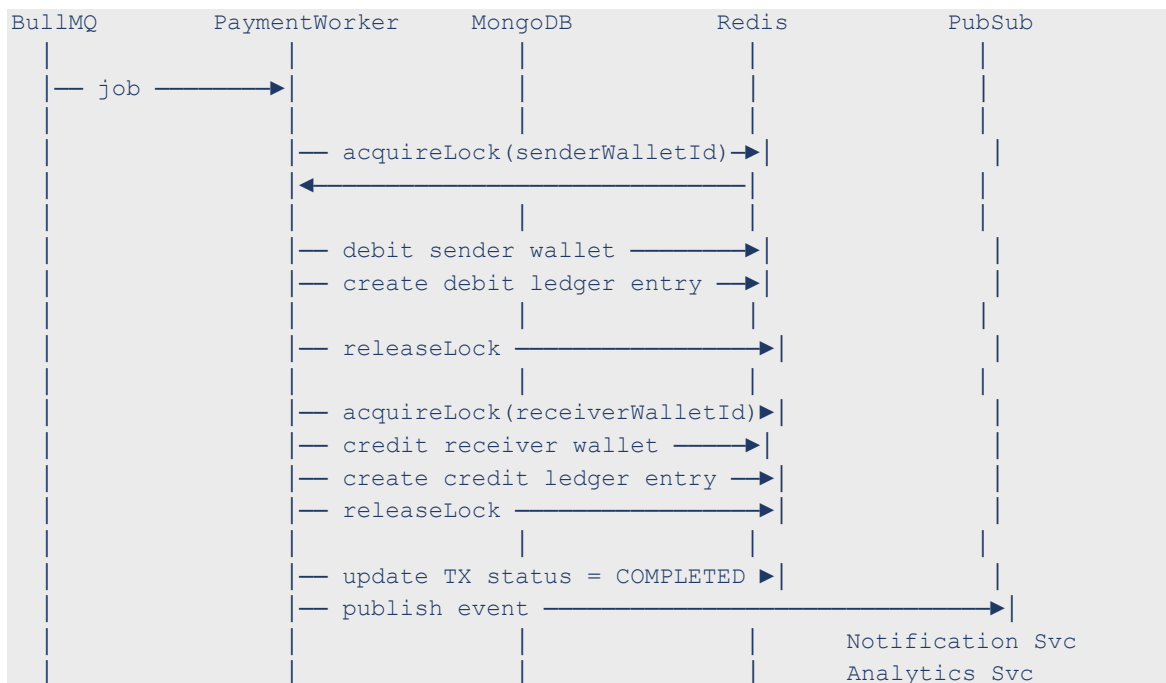


6.2 Request Flow: Transfer Initiation





6.3 Event Flow: Payment Processing



7. Service-by-Service Design

7.1 API Gateway

Responsibilities

- Single entry point for all external traffic
- JWT verification on every request except /auth routes
- Rate limiting per IP and per user
- Request routing to downstream services
- Request ID injection for distributed tracing

Internal Architecture

The API Gateway is an Express.js application with a chain of middleware executed in order:

1. Request ID middleware — generates a UUID and attaches it to req.requestId
2. Pino logger middleware — logs incoming request with method, path, requestId
3. Rate limiter middleware — checks Redis counter for the client IP
4. JWT verification middleware — validates Bearer token, injects req.user
5. Proxy router — forwards request to the appropriate downstream service

Failure Scenarios

- If JWT secret is invalid or expired: 401 Unauthorized
- If rate limit is exceeded: 429 Too Many Requests
- If downstream service is unavailable: 503 Service Unavailable with error body

7.2 Auth Service

API Boundaries

Endpoint	Description
POST /auth/register	Create new user account
POST /auth/login	Authenticate and return tokens
POST /auth/refresh	Exchange refresh token for new access token
POST /auth/logout	Invalidate current refresh token
GET /auth/me	Return current authenticated user profile

Token Architecture

- Access Token: JWT, signed with RS256, expires in 15 minutes. Stateless. Contains userId, email, iat, exp.
- Refresh Token: Cryptographically random UUID, stored in MongoDB with expiry of 7 days. One active refresh token per device session.

- Token Rotation: On every /auth/refresh call, the old refresh token is invalidated and a new one is issued. This prevents token reuse after theft.
- Token Blacklist: Revoked access tokens are stored in Redis with TTL equal to their remaining lifetime.

Password Security

- Passwords are hashed with bcrypt using cost factor 12 before storage
- Plaintext passwords are never logged or stored
- Failed login attempts are rate-limited

7.3 Wallet Service

Responsibilities

- Create wallets for authenticated users
- Return current balance
- Execute credit and debit operations with distributed locks
- Maintain ledger entries for every balance change

Critical Design: Distributed Locking

Before any debit or credit, the Wallet Service acquires a Redis-based distributed lock on the wallet ID. This prevents concurrent modifications to the same wallet.

```
Lock key: lock:wallet:{walletId}
TTL: 10 seconds
Retry: 3 attempts with 100ms backoff
```

If the lock cannot be acquired within the retry window, the operation fails with a 409 Conflict and the calling worker handles the retry.

Ledger Consistency

Every wallet operation writes a ledger entry atomically with the balance update. The ledger entry captures balanceBefore and balanceAfter, making the complete financial history auditable and rebuildable.

7.4 Transaction Service

Responsibilities

- Accept transfer requests from authenticated users
- Validate sender balance before enqueueing
- Enforce idempotency using Redis

- Create a PENDING transaction record
- Publish a job to the payment queue
- Return the transaction ID to the caller

Idempotency Flow

```
1. Client sends idempotencyKey header (UUID v4)
2. Transaction Service checks Redis: GET idempotency:{key}
3. If found: return cached response immediately (HTTP 200)
4. If not found: process request, store response in Redis with 24h TTL
5. On completion: mark idempotency key as processed
```

Balance Pre-Check

Before enqueueing, the Transaction Service reads the sender's current balance from the Wallet Service. If insufficient, it returns 400 immediately without creating a transaction. This is a soft check — the definitive balance check happens in the worker under a lock.

7.5 Payment Worker

Responsibilities

- Consume jobs from the payment queue
- Execute the full debit-credit sequence
- Handle retries with exponential backoff
- Execute rollback if credit fails after debit
- Publish completion events

Processing Sequence

6. Acquire distributed lock on sender wallet
7. Re-validate sender balance (definitive check under lock)
8. Debit sender wallet
9. Write debit ledger entry
10. Release sender lock
11. Acquire distributed lock on receiver wallet
12. Credit receiver wallet
13. Write credit ledger entry
14. Release receiver lock
15. Update transaction status to COMPLETED
16. Publish payment.completed event to Redis PubSub

Rollback Scenario

If step 7 (credit receiver) fails after step 3 (debit sender) has succeeded, the worker executes a compensating transaction: credit the sender back for the same amount, write a reversal ledger entry, and set transaction status to `ROLLED_BACK`.

7.6 Notification Service

Responsibilities

- Subscribe to payment.completed and payment.failed events via Redis PubSub
- Compose email messages for sender and receiver
- Deliver emails via Resend API
- Store notification record with delivery status
- Retry failed notifications independently

Design Principle

Notification failures NEVER affect payment outcomes. The Notification Service is a downstream consumer of events. It operates asynchronously. If it crashes, payments continue processing. When it recovers, it processes any missed events from Redis Streams.

7.7 Analytics Service

Responsibilities

- Subscribe to transaction events
- Aggregate total sent / received per user per month
- Expose analytics query APIs

Data Model

Analytics aggregates are stored per user per period (YYYY-MM). When a transaction event is consumed, the service performs an upsert on the aggregate document, incrementing totalSent or totalReceived and the transaction count.

Consistency Guarantee

Analytics are eventually consistent. There is no guarantee they reflect the absolute latest transaction. This is acceptable because analytics are read-only and non-financial.

8. Database Design

8.1 Users Collection

```
Collection: users

Fields:
  _id      : ObjectId (PK)
  name     : String, required
  email    : String, required, unique
  passwordHash : String, required
  status   : String, enum[active, suspended, deleted], default: active
  createdAt : Date, default: now
  updatedAt : Date

Indexes:
  { email: 1 } - unique, used for login lookup
  { status: 1 } - used for admin filtering (sparse)

Constraints:
  - email must be lowercase and trimmed before storage
  - passwordHash must never be returned in API responses
  - status transitions: active→suspended, active→deleted only
```

8.2 Wallets Collection

```
Collection: wallets

Fields:
  _id      : ObjectId (PK)
  userId   : ObjectId, ref: users, required
  balance  : Number (integer paisa), default: 0, min: 0
  currency : String, default: INR
  status   : String, enum[active, frozen, closed]
  version  : Number, default: 0 // optimistic lock counter
  createdAt : Date
  updatedAt : Date

Indexes:
  { userId: 1 } - unique, one wallet per user
  { status: 1 } - for operational queries

Constraints:
  - balance must never be stored as a float
  - balance must never go below 0
  - version is incremented on every update (optimistic lock)
```

8.3 Ledger Entries Collection

```
Collection: ledger_entries

Fields:
  _id          : ObjectId (PK)
  walletId     : ObjectId, ref: wallets
  transactionId : ObjectId, ref: transactions
  type         : String, enum[credit, debit]
  amount       : Number (paisa, positive integer)
  balanceBefore : Number
  balanceAfter  : Number
  description   : String
  createdAt     : Date (immutable)

Indexes:
  { walletId: 1, createdAt: -1 } - for paginated history queries
  { transactionId: 1 }          - for transaction detail lookup

Constraints:
  - Records are IMMUTABLE. No updates or deletes ever.
  - balanceAfter = balanceBefore + amount (credit)
  - balanceAfter = balanceBefore - amount (debit)
```

8.4 Transactions Collection

```
Collection: transactions

Fields:
  _id          : ObjectId (PK)
  idempotencyKey : String, unique
  senderId     : ObjectId, ref: users
  senderWalletId : ObjectId, ref: wallets
  receiverId    : ObjectId, ref: users
  receiverWalletId : ObjectId, ref: wallets
  amount       : Number (paisa)
  currency     : String
  status       : String, enum[PENDING, PROCESSING, COMPLETED, FAILED,
ROLLED_BACK]
  jobId        : String (BullMQ job ID)
  failureReason : String, optional
  createdAt     : Date
  updatedAt    : Date
  completedAt   : Date, optional

Indexes:
  { idempotencyKey: 1 }          - unique, idempotency dedup
  { senderId: 1, createdAt: -1 } - sender history pagination
  { receiverId: 1, createdAt: -1 } - receiver history pagination
  { status: 1 }                 - for operational monitoring
  { jobId: 1 }                  - worker lookup
```

8.5 Refresh Tokens Collection

```
Collection: refresh_tokens

Fields:
  _id      : ObjectId
  userId   : ObjectId, ref: users
  token    : String, unique (hashed UUID)
  deviceId : String (optional, for multi-device support)
  isRevoked : Boolean, default: false
  expiresAt : Date (7 days from creation)
  createdAt : Date

Indexes:
  { token: 1 } - unique, fast lookup on refresh
  { userId: 1 } - find all sessions for a user
  { expiresAt: 1 } - TTL index for auto-deletion
```

8.6 Notifications Collection

```
Collection: notifications

Fields:
  _id      : ObjectId
  userId   : ObjectId, ref: users
  type     : String, enum[payment_sent, payment_received, payment_failed]
  channel  : String, enum[email]
  payload  : Object { subject, body, transactionId }
  status   : String, enum[PENDING, SENT, FAILED]
  attempts : Number, default: 0
  createdAt : Date
  sentAt   : Date, optional

Indexes:
  { userId: 1, createdAt: -1 } - user notification history
  { status: 1 }                - for retry monitoring
```

8.7 Analytics Aggregates Collection

```
Collection: analytics_aggregates

Fields:
  _id      : ObjectId
  userId   : ObjectId
  period   : String (YYYY-MM format)
  totalSent : Number (paisa)
  totalReceived : Number (paisa)
  txCount   : Number
  lastUpdatedAt : Date
```

Indexes:

```
{ userId: 1, period: 1 } - unique compound, upsert target
```

9. Redis Architecture

Redis serves multiple distinct purposes in FinPay. Each use case has its own key namespace and TTL policy.

9.1 Key Naming Conventions

Namespace	: Pattern	: TTL
Rate limiting	: ratelimit:{ip}:{endpoint}	: 60s window
Access token BL	: blacklist:token:{jti}	: token remaining TTL
Refresh token store	: session:{userId}:{deviceId}	: 7 days
Distributed locks	: lock:wallet:{walletId}	: 10s max
Idempotency keys	: idempotency:{key}	: 24h
PubSub channel	: channel:payment.completed	: N/A (pub/sub)
PubSub channel	: channel:payment.failed	: N/A (pub/sub)
Stream key	: stream:payment.events	: permanent
BullMQ queue	: bull:{queueName}:*	: managed by BullMQ

9.2 Rate Limiting

Implemented with Redis INCR and EXPIRE. On each request:

17. INCR ratelimit:{ip}:{endpoint}
18. If count == 1: EXPIRE key to 60 seconds (first request in window)
19. If count > limit: reject with 429

Different limits apply to different endpoint categories. Auth endpoints have tighter limits (10/min) than read endpoints (100/min).

9.3 Token Blacklisting

When a user logs out, their access token's JWT ID (jti claim) is stored in Redis with TTL equal to the token's remaining lifetime. The API Gateway checks this on every request.

```
SET blacklist:token:{jti} 1 EX {remaining_seconds}
```

9.4 Distributed Locks

Implemented using Redis SET NX PX (set if not exists, with millisecond TTL). This is equivalent to the Redlock pattern for single-node Redis.

```
SET lock:wallet:{walletId} {lockToken} NX PX 10000
```

```
NX = only set if key does not exist
PX = TTL in milliseconds
10000 = 10 second max lock duration
```

The lock token is a unique value per lock acquisition. The release operation checks that the token matches before deleting, preventing accidental release of another holder's lock. A Lua script is used for atomic check-and-delete.

9.5 Idempotency Cache

When a transfer is successfully initiated, the response is serialized and stored in Redis:

```
SET idempotency:{key} {serializedResponse} EX 86400  
  
86400 = 24 hours in seconds
```

Subsequent requests with the same key return the cached response without re-processing.

9.6 BullMQ Queue

BullMQ uses Redis as its storage engine. All job state (waiting, active, completed, failed, delayed) is stored in Redis sorted sets and hashes under the bull: namespace. BullMQ manages this automatically.

9.7 Redis PubSub / Streams

Payment completion events are published on Redis PubSub channels. Notification Service and Analytics Service subscribe to these channels. For guaranteed delivery (if a subscriber restarts), Redis Streams provides a persistent event log with consumer group support.

```
PUBLISH channel:payment.completed {eventPayload}  
XADD stream:payment.events * eventType payment.completed payload {...}
```

10. Payment Processing Lifecycle

This section traces a complete money transfer from initiation to notification, step by step.

Scenario: User A transfers ₹200 (20000 paisa) to User B

Step 1 — Client Submits Transfer Request

```
POST /api/transfers
Authorization: Bearer {accessToken}
Idempotency-Key: a3f8e2d1-...

{
  receiverEmail: 'userb@example.com',
  amount: 20000,
  currency: 'INR'
}
```

Step 2 — API Gateway Processing

- Verifies JWT signature and expiry
- Checks access token is not in blacklist
- Checks rate limit for this user
- Injects requestId into headers
- Routes to Transaction Service

Step 3 — Transaction Service: Idempotency Check

- Reads Idempotency-Key from request header
- Checks Redis: GET idempotency:a3f8e2d1-...
- If found: return cached 200 response immediately
- If not found: proceed to validation

Step 4 — Transaction Service: Validation

- Resolves receiver by email — finds User B's wallet
- Validates amount > 0
- Pre-checks sender balance — calls Wallet Service
- If balance < 20000: return 400 Insufficient Balance

Step 5 — Transaction Service: Record Creation

- Creates Transaction document with status: PENDING
- Stores idempotency key in Redis with 24h TTL
- Enqueues job in BullMQ: { transactionId, senderWalletId, receiverWalletId, amount }
- Returns 202 Accepted: { transactionId, status: PENDING }

Step 6 — Payment Worker: Job Pickup

- BullMQ delivers job to an available Payment Worker instance
- Worker updates Transaction status to PROCESSING

Step 7 — Payment Worker: Debit Sender

```
1. SET lock:wallet:{senderWalletId} {lockToken} NX PX 10000
2. Read sender wallet - verify balance >= 20000 (definitive check)
3. Update: wallet.balance -= 20000, wallet.version++
4. Create ledger entry: type=debit, amount=20000, balanceBefore=100000,
balanceAfter=80000
5. DEL lock:wallet:{senderWalletId} (atomic Lua script)
```

Step 8 — Payment Worker: Credit Receiver

```
1. SET lock:wallet:{receiverWalletId} {lockToken} NX PX 10000
2. Update: wallet.balance += 20000, wallet.version++
3. Create ledger entry: type=credit, amount=20000, balanceBefore=50000,
balanceAfter=70000
4. DEL lock:wallet:{receiverWalletId}
```

Step 9 — Transaction Completion

- Transaction status updated to COMPLETED, completedAt set
- Worker publishes event to Redis PubSub: channel:payment.completed
- Worker also appends to Redis Stream: stream:payment.events
- Job marked as complete in BullMQ

Step 10 — Notification Service

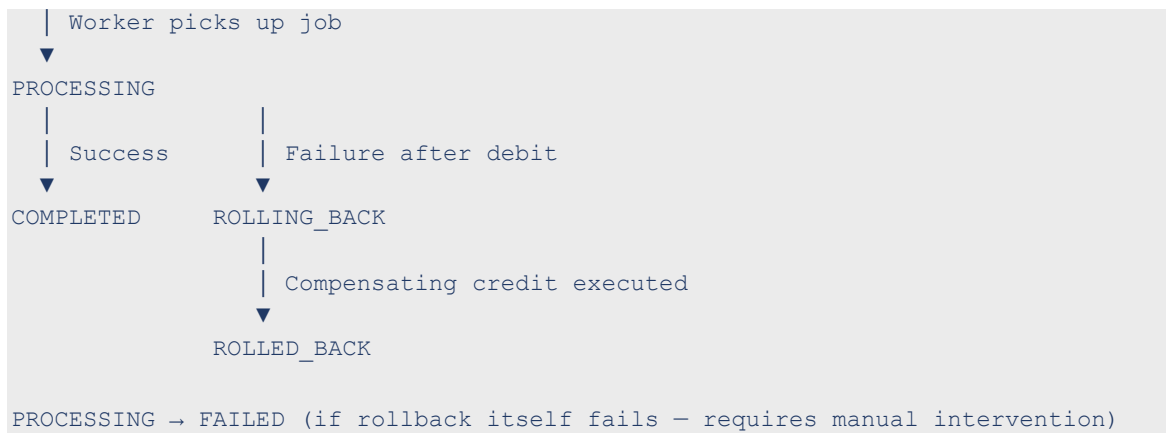
- Receives payment.completed event
- Composes debit email for User A: 'You sent ₹200 to User B'
- Composes credit email for User B: 'You received ₹200 from User A'
- Delivers both emails via Resend API
- Stores notification records with status SENT

Step 11 — Analytics Service

- Receives payment.completed event
- Upserts User A's aggregate for current period: totalSent += 20000, txCount++
- Upserts User B's aggregate for current period: totalReceived += 20000, txCount++

State Transition Diagram

```
PENDING
|
```

11. Distributed Systems Deep Dive

11.1 Distributed Locking

The wallet balance is a shared mutable resource. Without locking, two concurrent transfers from the same sender can both read the current balance (e.g. ₹1000), both determine they have sufficient funds, and both proceed to deduct — resulting in a net deduction of ₹400 from what should have been ₹1000, potentially creating a negative balance.

Redis SET NX PX provides mutual exclusion. The lock key is unique to each wallet. Only one holder can hold the lock at a time. The TTL ensures the lock is released even if the holder crashes.

Why Redis and not MongoDB transactions?

MongoDB multi-document ACID transactions are available but incur significant overhead and require replica sets.

Redis lock acquisition is a single O(1) operation, making it far faster for high-frequency wallet operations.

The combination of Redis locks + MongoDB single-document atomicity gives strong consistency with excellent performance.

11.2 Race Conditions

```
Without lock:
T=0ms Worker1 reads: balance=1000
T=1ms Worker2 reads: balance=1000
T=2ms Worker1 writes: balance=800 (deducted 200)
T=3ms Worker2 writes: balance=700 (deducted 300, from stale read)
Result: 500 deducted, but balance shows 700. Inconsistency!

With lock:
T=0ms Worker1 acquires lock
T=0ms Worker2 tries - blocked
T=1ms Worker1 reads: balance=1000
T=2ms Worker1 writes: balance=800
T=2ms Worker1 releases lock
T=3ms Worker2 acquires lock
T=4ms Worker2 reads: balance=800
T=5ms Worker2 writes: balance=500
Result: Correct.
```

11.3 Idempotency

A client may not know if a transfer was processed because a network failure occurred before they received the response. They will retry. Without idempotency, this creates a duplicate transfer.

Idempotency is enforced at the Transaction Service by requiring clients to send a unique Idempotency-Key header (a UUID they generate). The service stores the response mapped to

this key in Redis for 24 hours. Any retry with the same key returns the cached response without re-processing.

- The key is generated by the client, not the server
- The key is tied to a specific intent, not just a user
- The 24-hour window covers realistic retry scenarios

11.4 Eventual Consistency

FinPay uses eventual consistency for non-financial data. Wallet balances are strongly consistent (under distributed lock). Analytics aggregates are eventually consistent — they are updated asynchronously after payment completion and may lag behind by seconds.

This tradeoff is intentional. Requiring real-time analytics would add latency to the payment critical path. Eventual consistency is acceptable because analytics data is never used to make financial decisions within the system.

11.5 Retry Mechanisms and Backoff

Payment jobs that fail are automatically retried by BullMQ. Retries use exponential backoff with jitter to avoid thundering herd:

```
Attempt 1: immediate
Attempt 2: 2^1 * 1000ms = 2s + random jitter
Attempt 3: 2^2 * 1000ms = 4s + random jitter
Attempt 4: 2^3 * 1000ms = 8s + random jitter
Attempt 5: 2^4 * 1000ms = 16s + random jitter
After 5 attempts: move to Dead Letter Queue
```

Retries are safe because the worker checks idempotency before re-executing. A partially completed job (sender debited but receiver not yet credited) detects this state and skips the already-completed debit step.

11.6 Saga Pattern

A payment transfer is a distributed transaction spanning two wallet operations. The Saga pattern manages this as a sequence of local transactions, each with a compensating transaction for rollback.

```
Forward Steps:
  T1: Debit sender → Compensation C1: Credit sender (reversal)
  T2: Credit receiver → Compensation: (none needed if T1 already compensated)

If T1 succeeds and T2 fails:
  Execute C1: credit sender back
  Set transaction status: ROLLED_BACK

If T1 fails:
  No compensation needed (nothing was changed)
  Set transaction status: FAILED
```

This is a choreography-based Saga (no central orchestrator). The Payment Worker is responsible for executing the full saga sequence.

11.7 Dead Letter Queues

Jobs that exhaust all retry attempts are moved to a Dead Letter Queue (DLQ). DLQ jobs are not lost — they are stored in Redis and visible in Bull Board for manual inspection and reprocessing.

A job in the DLQ represents a transfer that is in an uncertain state. Operations engineers must investigate: was the sender debited? Was the receiver credited? The ledger entries provide the evidence needed to determine the true state and decide whether to retry, refund, or mark as permanently failed.

12. Failure Handling

Failure Scenario	What Happens	Recovery Strategy
Sender wallet not found	Transaction Service returns 404	Client retries with correct data
Insufficient balance (soft check)	Transaction Service returns 400	No job created, no retry needed
Redis lock cannot be acquired (payment worker)	Worker throws LockAcquisitionError	BullMQ retries job with backoff
Definitive balance insufficient (under lock)	Worker throws InsufficientFundsError	Transaction set to FAILED, no retry
MongoDB write failure (debit)	Worker throws, nothing changed	BullMQ retries — safe, idempotent
MongoDB write failure (credit, after debit)	Worker detects partial state	Compensating credit executed, ROLLED_BACK
Redis unavailable (locks)	All wallet operations fail	Circuit breaker trips; payments suspended
BullMQ worker crash mid-job	Job remains in active state	BullMQ moves to failed after stall timeout
Notification Service down	Events accumulate in Redis Stream	Service resumes, processes backlog
Resend API failure	Email not delivered	Notification Service retries independently
Duplicate transfer request	Idempotency key match found	Cached response returned, no new transfer
Analytics Service down	Events buffered in Redis Stream	Service catches up on restart

12.1 Partial Failure Detection

The Payment Worker detects partial completion state by checking the transaction's ledger entries on job start. If a debit entry exists but no credit entry exists, the job knows compensation is needed. This check is performed before acquiring any locks.

12.2 Circuit Breaker (Manual, No Framework)

In the current implementation, Redis unavailability causes immediate failures. The failure threshold for Redis is zero — any connection failure suspends lock-dependent operations. A proper circuit breaker pattern would track failure rates and open the circuit automatically. This is listed as a future improvement.

13. Security Architecture

13.1 JWT Design

- Algorithm: RS256 (asymmetric). Private key signs tokens; public key verifies. This allows the API Gateway to verify tokens without having the private key.
- Access Token Lifetime: 15 minutes. Short-lived to limit damage from theft.
- Claims: { sub: userId, email, iat, exp, jti }. The jti (JWT ID) enables blacklisting.
- Storage Recommendation: Memory only in the client. Not localStorage (XSS risk).

13.2 Refresh Token Rotation

Every call to POST /auth/refresh performs atomic rotation:

20. Validate old refresh token exists in MongoDB and is not revoked
21. Verify token has not expired
22. Generate new refresh token
23. Mark old token as isRevoked: true
24. Store new token in MongoDB
25. Return new access token + new refresh token

If a stolen refresh token is used by an attacker, and the legitimate user also tries to refresh, the second refresh attempt finds the first token already revoked. Both sessions are invalidated. The user must log in again.

13.3 Rate Limiting

Endpoint Group	Limit
POST /auth/register	5 requests per hour per IP
POST /auth/login	10 requests per minute per IP
POST /transfers	20 requests per minute per user
GET endpoints	100 requests per minute per user
POST /auth/refresh	30 requests per minute per IP

13.4 Password Security

- bcrypt with cost factor 12 (approximately 250ms per hash on modern hardware)
- Password minimum: 8 characters
- Passwords are never logged at any level
- The passwordHash field is explicitly excluded from all API responses

13.5 Transaction Safety

- Distributed locks prevent concurrent modifications to wallet balances
- Idempotency keys prevent duplicate transfers
- Balance is pre-checked before queue entry AND re-checked under lock before debit
- Ledger entries are immutable — financial history cannot be altered
- All wallet balance updates use integer paisa — no floating point

14. API Design

14.1 Auth Endpoints

POST /api/auth/register

Body: { name, email, password }

Response 201: { userId, name, email }

Errors: 400 validation error | 409 email already exists

POST /api/auth/login

Body: { email, password }

Response 200: { accessToken, refreshToken, user: { id, name, email } }

Errors: 400 validation | 401 invalid credentials | 429 rate limited

POST /api/auth/refresh

Body: { refreshToken }

Response 200: { accessToken, refreshToken }

Errors: 401 invalid/expired token | 429 rate limited

POST /api/auth/logout

Headers: Authorization: Bearer {accessToken}

Body: { refreshToken }

Response 204: No Content

Errors: 401 unauthorized

14.2 Wallet Endpoints

POST /api/wallets

Headers: Authorization required

Response 201: { walletId, balance: 0, currency: INR, status: active }

Errors: 409 wallet already exists | 401 unauthorized

GET /api/wallets/me

Headers: Authorization required

Response 200: { walletId, balance, currency, status, createdAt }

Errors: 404 no wallet | 401 unauthorized

14.3 Transfer Endpoints

POST /api/transfers

Headers: Authorization: Bearer | Idempotency-Key: {uuid}

Body: { receiverEmail, amount, currency }

Response 202: { transactionId, status: PENDING, message }

Errors: 400 invalid amount | 400 insufficient balance | 404 receiver not found | 409 same-user transfer | 422 idempotency key reused with different params

GET /api/transfers/:transactionId

Headers: Authorization required

Response 200: { transactionId, status, amount, sender, receiver, createdAt, completedAt }

Errors: 404 not found | 403 not your transaction

GET /api/transfers?page=1&limit=20

Headers: Authorization required

Response 200: { transactions: [...], total, page, limit }

Each item: { id, direction: sent|received, amount, counterparty, status, createdAt }

14.4 Analytics Endpoints

GET /api/analytics/summary?period=2024-11

Headers: Authorization required

Response 200: { period, totalSent, totalReceived, txCount }

Note: Eventually consistent — may lag by seconds

15. Folder Structure

```

finpay/
├── packages/
│   ├── shared/                # Shared utilities across all services
│   │   └── logger/            # Pino logger factory with correlation ID
│   └── support
│       ├── errors/            # Custom error classes (AppError, NotFoundError,
│       │   └── errorHandler)  etc.)
│       ├── middleware/        # Express middleware (auth, requestId,
│       │   └── redis/          # Redis client factory, lock utilities
│       │       └── validators/  # Joi/Zod schema validators
│       └── api-gateway/        # Entry point - routing, JWT, rate limiting
│           ├── src/
│           │   ├── config/
│           │   ├── middleware/
│           │   ├── routes/      # Proxy route definitions
│           │   └── app.js
│           └── Dockerfile
│
│   ├── auth-service/
│       ├── src/
│       │   ├── config/
│       │   ├── controllers/
│       │   ├── services/        # AuthService, TokenService
│       │   ├── models/          # User, RefreshToken Mongoose models
│       │   ├── routes/
│       │   └── app.js
│       └── Dockerfile
│
│   ├── wallet-service/
│       ├── src/
│       │   ├── controllers/
│       │   ├── services/        # WalletService, LedgerService, LockService
│       │   ├── models/          # Wallet, LedgerEntry Mongoose models
│       │   ├── routes/
│       │   └── app.js
│       └── Dockerfile
│
│   ├── transaction-service/
│       ├── src/
│       │   ├── controllers/
│       │   └── services/        # TransactionService, IdempotencyService,
│       │   QueueService
│       │       ├── models/      # Transaction Mongoose model
│       │       ├── queues/      # BullMQ queue definitions
│       │       ├── routes/
│       │       └── app.js
│       └── Dockerfile
│
│   └── payment-worker/

```

```
|
|
|   └─ src/
|       ├── processors/    # PaymentProcessor – the core saga executor
|       ├── services/     # WalletClient (HTTP calls to wallet-service)
|       ├── events/       # Event publisher
|       └─ worker.js       # BullMQ Worker bootstrap
|   └─ Dockerfile
|
|   └─ notification-service/
|       ├── src/
|           ├── subscribers/ # Redis PubSub listener
|           ├── services/   # NotificationService, EmailService (Resend)
|           ├── models/     # Notification Mongoose model
|           └─ app.js
|       └─ Dockerfile
|
|   └─ analytics-service/
|       ├── src/
|           ├── subscribers/ # Redis Stream consumer
|           ├── services/   # AggregationService
|           ├── models/     # AnalyticsAggregate Mongoose model
|           ├── routes/     # Analytics query API
|           └─ app.js
|       └─ Dockerfile
|
| └─ docker-compose.yml
| └─ docker-compose.dev.yml
| └─ .env.example
| └─ package.json          # Monorepo root (npm workspaces)
| └─ README.md
```

16. Docker Architecture

16.1 docker-compose.yml Services

Service	Image / Notes
mongo	mongo:7.0 — primary database
redis	redis:7.2-alpine — cache, locks, queues, pubsub
api-gateway	Custom Dockerfile, port 3000:3000, depends on auth-service
auth-service	Custom Dockerfile, port 3001 (internal only)
wallet-service	Custom Dockerfile, port 3002 (internal only)
transaction-service	Custom Dockerfile, port 3003 (internal only)
payment-worker	Custom Dockerfile, no exposed port (worker only)
notification-service	Custom Dockerfile, no exposed port
analytics-service	Custom Dockerfile, port 3006 internal for API
bull-board	Custom or bullmq/bull-board image, port 3010:3010 (monitoring UI)

16.2 Network Design

```
Network: finpay_internal (bridge)
```

```
All services communicate via service name DNS resolution.
Only api-gateway and bull-board expose ports to the host.
MongoDB and Redis are not exposed to the host in production.
```

```
Service discovery:
  http://auth-service:3001
  http://wallet-service:3002
  http://transaction-service:3003
  mongodb://mongo:27017/finpay
  redis://redis:6379
```

16.3 Startup Dependencies

```
mongo      (no deps)
redis      (no deps)
auth-service    → depends_on: mongo, redis
wallet-service  → depends_on: mongo, redis
transaction-service → depends_on: mongo, redis, wallet-service
api-gateway    → depends_on: auth-service, wallet-service, transaction-
service
payment-worker → depends_on: mongo, redis, wallet-service
```

```
notification-service → depends_on: mongo, redis  
analytics-service   → depends_on: mongo, redis
```

16.4 Health Checks

Every service exposes GET /health returning { status: ok, service: name, timestamp }. Docker health checks use this endpoint. Downstream services only start after upstream health checks pass.

17. Logging and Observability

17.1 Structured Logging with Pino

Every service uses Pino for JSON-structured logging. Every log line includes:

Field	Description
timestamp	ISO 8601 UTC
level	trace debug info warn error
service	Service name (e.g. auth-service)
requestId	UUID injected by API Gateway, propagated via headers
userId	Authenticated user ID if available
method / path	HTTP method and route
statusCode	Response code
durationMs	Request processing time
error	Error message and stack for error-level logs

17.2 Request Tracing

1. API Gateway generates requestId: UUID v4
2. Injects as X-Request-ID header in forwarded request
3. Each downstream service reads X-Request-ID and includes it in all log lines
4. Worker propagates requestId from job data to its log lines
5. All logs for a single user transfer share the same requestId

17.3 Queue Monitoring — Bull Board

Bull Board provides a web UI at :3010/admin/queues showing:

- Waiting, active, completed, failed, and delayed job counts
- Job details including payload, attempt count, and error messages
- Dead Letter Queue contents
- Ability to retry or remove failed jobs

17.4 Key Metrics to Track

- Payment queue depth (jobs waiting)
- Payment processing rate (jobs/second)
- DLQ size (permanently failed jobs)
- Wallet lock acquisition failures (rate)
- Auth endpoint error rate (spikes indicate brute force)

- MongoDB operation latency (p50, p95, p99)

18. Scaling Strategy

18.1 Horizontal Service Scaling

Every service is stateless and can run multiple instances. The API Gateway routes to any instance of a downstream service using round-robin DNS (Docker Compose) or a load balancer (Kubernetes).

18.2 Payment Worker Scaling

Payment Workers are the most scalable component. Because they are queue consumers, adding more worker instances increases throughput linearly. BullMQ's concurrency setting also allows a single worker process to handle multiple jobs simultaneously.

```
Single worker, concurrency 1: ~50 TPS
Single worker, concurrency 10: ~500 TPS
5 workers, concurrency 10: ~2500 TPS (estimate)

Bottleneck at scale: Redis I/O, then MongoDB write throughput
```

18.3 Redis Bottlenecks

- Redis is single-threaded for commands but highly optimized for low-latency ops
- At very high scale, Redis Cluster shards data across multiple nodes
- Lock operations are $O(1)$ and sub-millisecond in practice
- BullMQ queue operations scale well with Redis Cluster

18.4 MongoDB Scaling

- Read replicas handle analytics and history queries without burdening the primary
- Wallet and transaction writes go to the primary — sharding by `userId` is the next step
- Ledger collection grows unboundedly — archival strategy needed for production
- Appropriate indexes (defined in Section 8) are essential to maintain query performance

19. Future Improvements

Improvement	Value / Rationale
Apache Kafka	Replace Redis PubSub with Kafka for durable, replayable, high-throughput event streaming. Kafka retains events permanently and supports consumer groups across multiple services.
Kubernetes	Move from Docker Compose to Kubernetes for auto-scaling, self-healing, and rolling deployments. Horizontal Pod Autoscaler scales payment workers based on queue depth.
Circuit Breakers	Implement circuit breakers (e.g., opossum library) around downstream service calls and Redis operations. Prevent cascade failures when dependencies are degraded.
Event Sourcing	Store wallet balance as a stream of events (all ledger entries) and derive current balance by replaying events. Enables perfect audit trail and point-in-time balance reconstruction.
CQRS	Separate read models from write models. Write path updates wallet; read path queries pre-computed views. Enables independent scaling of reads and writes.
OpenTelemetry	Replace manual requestId propagation with OpenTelemetry distributed tracing. Enables Jaeger/Zipkin trace visualization across all services.
Prometheus + Grafana	Export metrics from every service (via prom-client), scrape with Prometheus, visualize with Grafana dashboards for queue depth, error rates, and latency.
Outbox Pattern	Eliminate the gap between MongoDB write and queue publish by writing the job to a MongoDB 'outbox' collection atomically with the transaction record, then publishing from the outbox. Prevents lost messages.

20. Development Roadmap

Phase 1 — MVP

Goals & Deliverables

Goals: Working auth, wallet, and basic synchronous transfer

Deliverables:

- User registration and login with JWT
- Wallet creation and balance view
- Synchronous transfer (no queue yet)
- MongoDB schemas for all core entities
- Docker Compose with Mongo + Redis

Architecture: Monolith-style single service, no queue

Learning: MongoDB schema design, JWT auth, bcrypt, Docker basics

Phase 2 — Reliable Payments

Goals & Deliverables

Goals: Correct concurrency handling and idempotency

Deliverables:

- Distributed locking on wallet operations
- Idempotency key enforcement
- Refresh token rotation
- Rate limiting middleware
- Ledger entry writes on every balance change

Architecture: Services split; Redis used for locks, rate limiting, idempotency

Learning: Distributed locks, race conditions, Redis, idempotency patterns

Phase 3 — Async Processing

Goals & Deliverables

Goals: Queue-based payment processing with retries and rollbacks

Deliverables:

- BullMQ payment queue
- Payment Worker service
- Saga pattern for debit-credit-rollback
- Dead Letter Queue configuration
- Bull Board monitoring UI

Architecture: Transaction Service enqueues; Payment Worker executes
Learning: Queue architecture, saga pattern, retry backoff, DLQ, worker design

Phase 4 — Analytics & Notifications

Goals & Deliverables

Goals: Event-driven downstream consumers

Deliverables:

- Redis PubSub event publishing from Payment Worker
- Notification Service subscribing and sending emails via Resend
- Analytics Service consuming events and aggregating
- Analytics query API
- Notification history stored in MongoDB

Architecture: Payment Worker publishes; Notification + Analytics consume independently

Learning: Event-driven architecture, pub/sub, eventual consistency, Redis Streams

Phase 5 — Production Readiness

Goals & Deliverables

Goals: Observability, security hardening, operational excellence

Deliverables:

- Structured logging with Pino and requestId propagation
- Centralized error handling and error codes
- Health check endpoints on every service
- Input validation on all endpoints (Joi or Zod)
- .env.example and configuration management
- README and API documentation
- Optional: OpenTelemetry tracing spike

Architecture: All 7 services containerized, orchestrated via Docker Compose

Learning: Observability, config management, production patterns, documentation

End of Document